

Shiv Nadar University Chennai

CS3807 - Deep Learning Laboratory

Experiment 7

Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Name	Roll number	Batch	Semester	Academic year
Joshua	24110085	2	V	2026-27

B.Tech Artificial Intelligence & Data Science

1. Objective

To reconstruct MNIST images, remove image noise, compare latent representations and generate new digit images.

2. Dataset and setup

MNIST images were scaled from [0, 255] to [0, 1]. The fixed split contained 10,000 training images, 2,000 validation images and 2,000 images from the official test set. Test images were not used for training or model selection. [1]

Setting	Recorded value
Input / output	1 x 28 x 28; 784 values after flattening
Optimizer	Adam; learning rate 0.001
Training	20 epochs; batch size 128; seed 42
Final training batch	16 images
Execution	PyTorch 2.10.0+cpu; CPU; 4 threads; float32
SSIM	Mean over test images; data_range = 1; 7 x 7 window
Checkpoint	Final epoch for every model

3. Method and model structure

Clean images were reconstruction targets. Denoising models received fixed noisy inputs, with Gaussian noise clipped to [0, 1] or salt-and-pepper pixels set to 0 or 1. Labels were used only for visual analysis.

MSE = $\text{mean}((x - \text{reconstruction})^2)$; MAE = $\text{mean}(|x - \text{reconstruction}|)$. SSIM measured structural similarity. [2]

The VAE sampled $z = \text{mean} + \exp(0.5 \log \text{variance}) \times \text{epsilon}$, with epsilon from $N(0, I)$. Its loss was BCE summed over pixels + $\beta \times \text{KL}$, where $\text{KL} = -0.5 \sum (1 + \log \text{variance} - \text{mean}^2 - \text{variance})$. [3]

Model	Architecture
FC-AE	784 - 128 - 32 - d - 32 - 128 - 784; ReLU hidden layers.
CAE / DAE	Conv32 - Pool2 - Conv64 - Pool2 - Conv64 - Up2 - Conv32 - Up2 - Conv1.
Vector CAE	The 64 x 7 x 7 pooled map was projected to d values and expanded before decoding.
VAE	Two stride-2 convolutions (32, 64), Flatten, direct mean/log-variance heads; dense and transposed-convolution decoder.

AE and CAE losses were pixel-mean BCE. Sigmoid outputs were implemented through logits-based BCE during training. VAE image metrics used the posterior mean; VAE objectives used one seeded posterior sample per image.

Source: Joshua-Divide/LAB | Notebook: Experiment_7.ipynb. Full code appears in the appendix.

4. Measured results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC-AE (d = 16)	0.027172	0.069386	0.692002	211,040	4.51
CAE	0.002742	0.015532	0.974080	74,497	76.86
DAE (sigma = 0.2)	0.004493	0.021158	0.948878	74,497	75.24
VAE (d = 2)	0.045632	0.106545	0.492553	139,461	104.95

FC-AE, CAE and VAE received clean images. DAE received Gaussian-noisy images at sigma = 0.2. Training time includes validation, but excludes final test evaluation.

Same clean-input comparison

Model	MSE	MAE	SSIM
FC_16	0.027172	0.069386	0.692002
CAE	0.002742	0.015532	0.974080
DAE_G02	0.003273	0.017009	0.965008
VAE_2	0.045632	0.106545	0.492553

VAE losses and reconstruction metrics

Model	Recon. BCE	KL	Total	MSE	MAE	SSIM
VAE_2	155.082552	5.419465	160.502017	0.045632	0.106545	0.492553
VAE_8	99.202801	15.919328	115.122128	0.019473	0.053841	0.786753
VAE_beta4	160.805074	3.433870	174.540554	0.046787	0.111755	0.464868

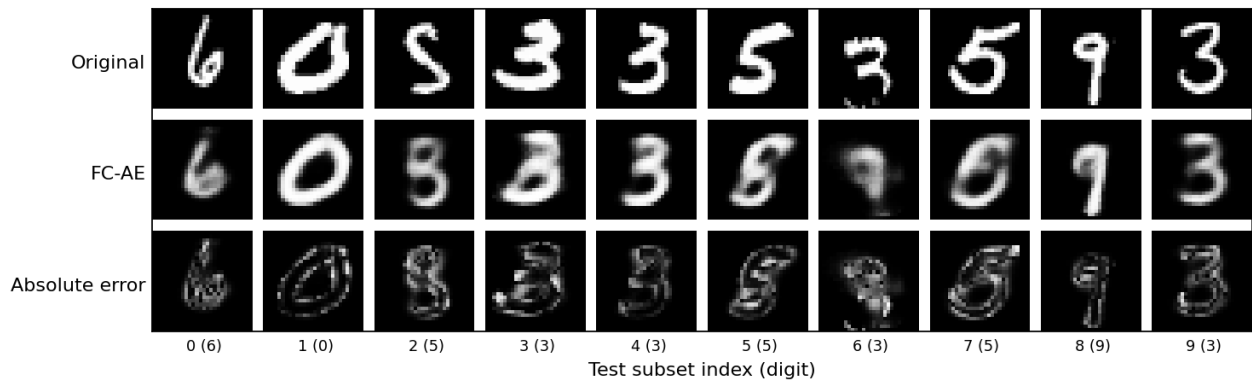
Beta = 1 for VAE_2 and VAE_8; beta = 4 for VAE_beta4. Total = reconstruction + beta x KL. VAE totals are not directly comparable with pixel-mean AE BCE.

All 16 trained configurations

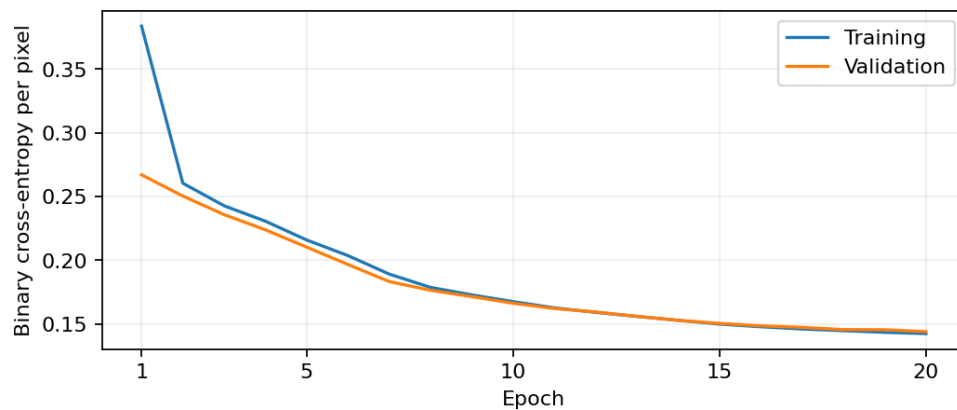
Model	MSE	MAE	SSIM	Parameters	Time (s)
FC_2	0.067606	0.150665	0.177235	210,130	5.30
FC_8	0.028763	0.072645	0.681657	210,520	5.66
FC_16	0.027172	0.069386	0.692002	211,040	4.51
FC_32	0.025608	0.066786	0.706838	212,080	4.87
CAE	0.002742	0.015532	0.974080	74,497	76.86
CAE_4	0.032648	0.080543	0.658076	102,725	85.70
CAE_8	0.020067	0.053428	0.789090	127,817	87.09
CAE_16	0.010224	0.033164	0.894051	178,001	87.74
CAE_32	0.005971	0.023702	0.938890	278,369	91.40
CAE_transpose	0.002303	0.014238	0.977181	89,057	58.95
DAE_G02	0.004493	0.021158	0.948878	74,497	75.24
DAE_G03	0.006552	0.026186	0.923018	74,497	77.37
DAE_SP10	0.005150	0.022092	0.945470	74,497	75.03
VAE_2	0.045632	0.106545	0.492553	139,461	104.95
VAE_8	0.019473	0.053841	0.786753	195,921	98.55
VAE_beta4	0.046787	0.111755	0.464868	139,461	101.12

DAE_G02 and DAE_G03 used Gaussian sigma 0.2 and 0.3. DAE_SP10 used salt-and-pepper probability 0.10. The remaining configurations received clean inputs. All values are from the same fixed-seed execution.

5. Basic reconstruction and loss

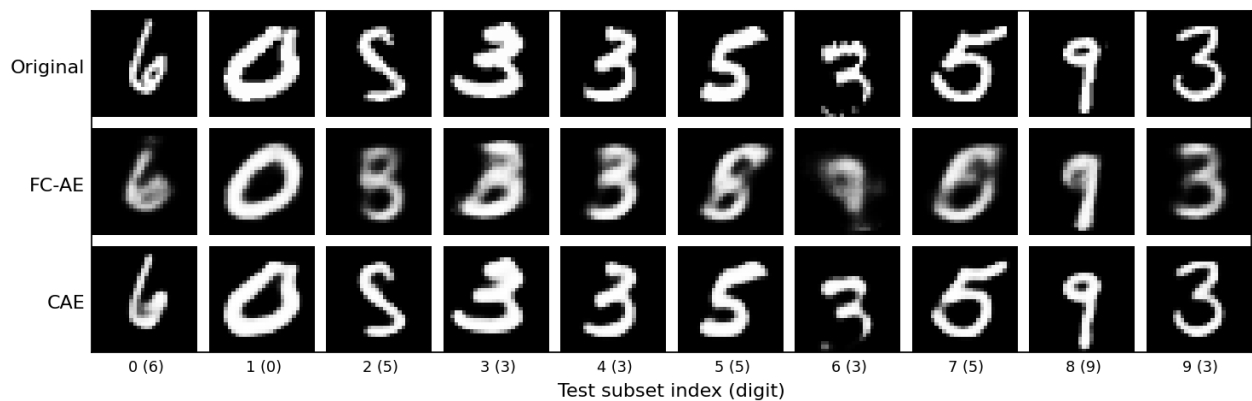


Inference: The FC-AE retained several digit shapes, but blurred thin strokes and changed some 5s into 8-like shapes. The error row marks these lost details.



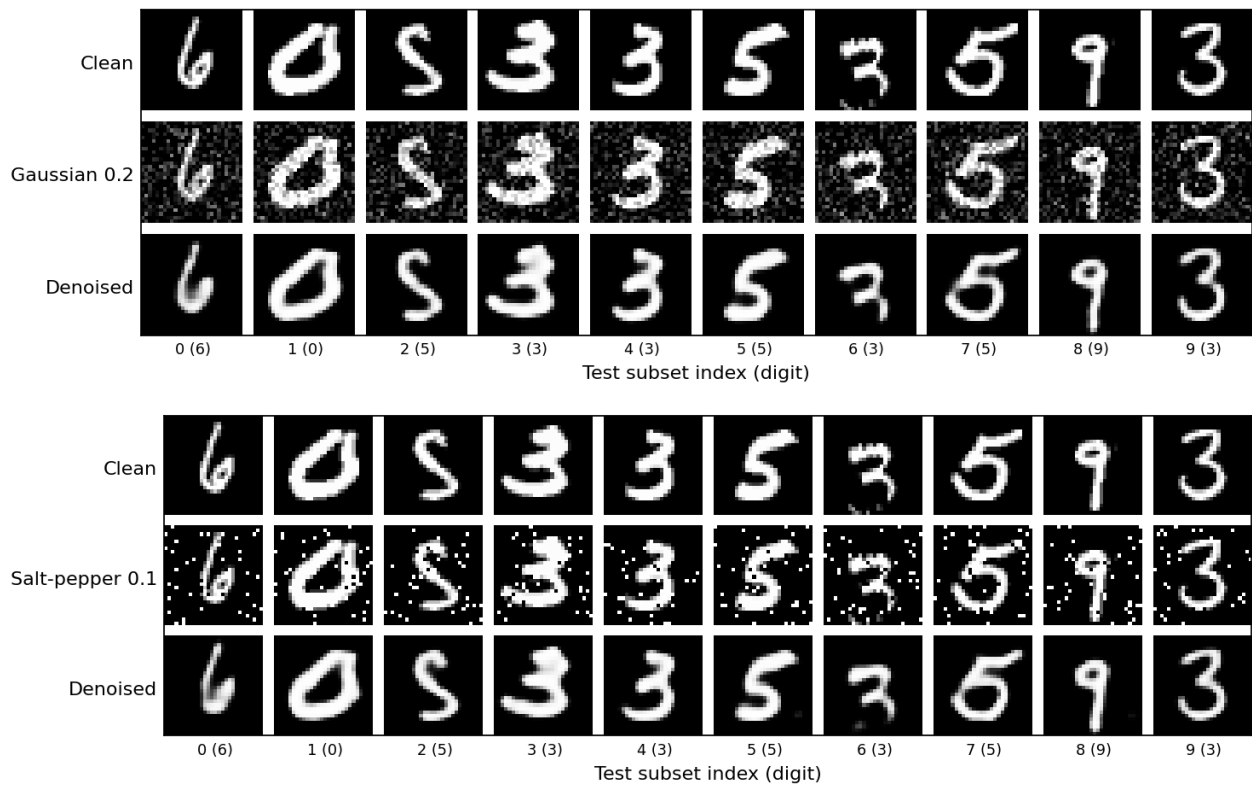
Inference: Training BCE fell from 0.38368 to 0.14242, and validation BCE from 0.26702 to 0.14401. Both curves stayed close, with no clear overfitting during these 20 epochs.

Fully connected and convolutional reconstruction



Inference: The CAE kept sharper strokes and lowered MSE from 0.02717 to 0.00274. Its 3,136-value feature map was larger than the FC-AE's 16-value code, so the comparison also changes bottleneck size.

6. Denoising

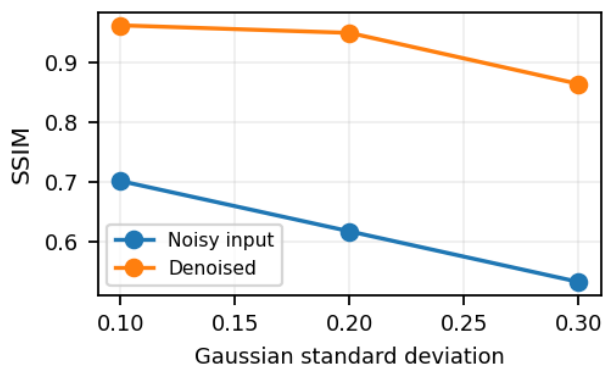
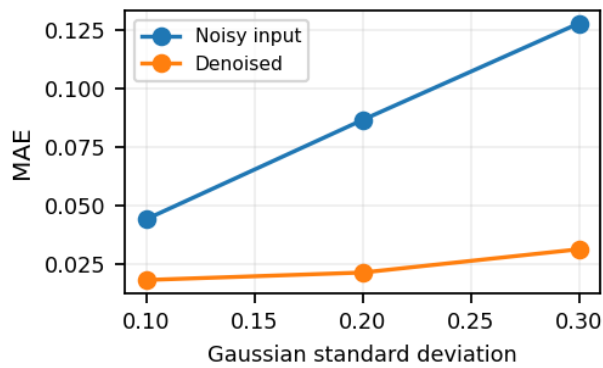
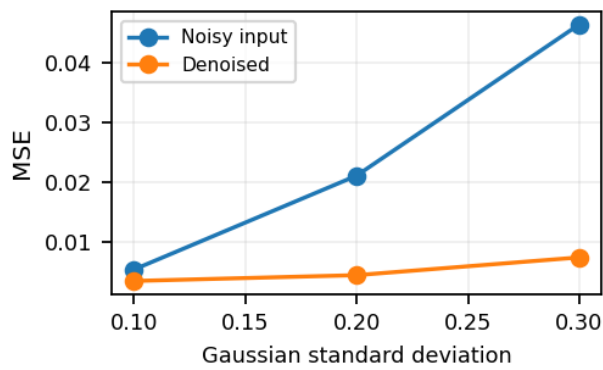


Noise	Level	Output	MSE	MAE	SSIM
Gaussian	0.10	Noisy input	0.005422	0.044053	0.700865
Gaussian	0.10	Denoised	0.003518	0.017971	0.961530
Gaussian	0.20	Noisy input	0.021167	0.086510	0.616586
Gaussian	0.20	Denoised	0.004493	0.021158	0.948878
Gaussian	0.30	Noisy input	0.046530	0.127839	0.531867
Gaussian	0.30	Denoised	0.007470	0.031121	0.863364
Salt-pepper	0.05	Noisy input	0.024017	0.024944	0.725749
Salt-pepper	0.05	Denoised	0.004240	0.019763	0.957220
Salt-pepper	0.10	Noisy input	0.048041	0.049896	0.598735
Salt-pepper	0.10	Denoised	0.005150	0.022092	0.945470
Salt-pepper	0.20	Noisy input	0.096425	0.100154	0.466041
Salt-pepper	0.20	Denoised	0.008290	0.029394	0.899596

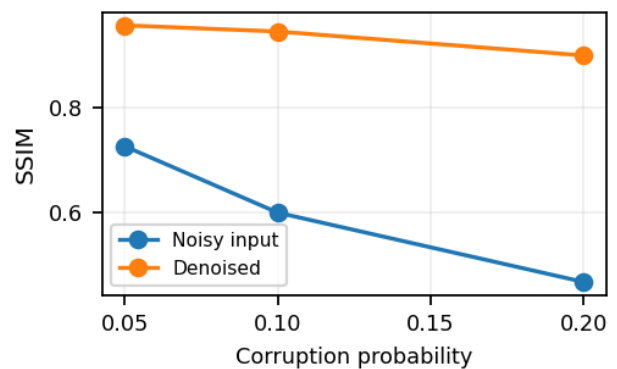
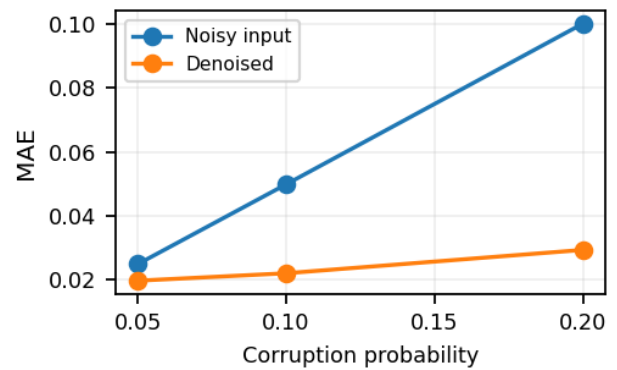
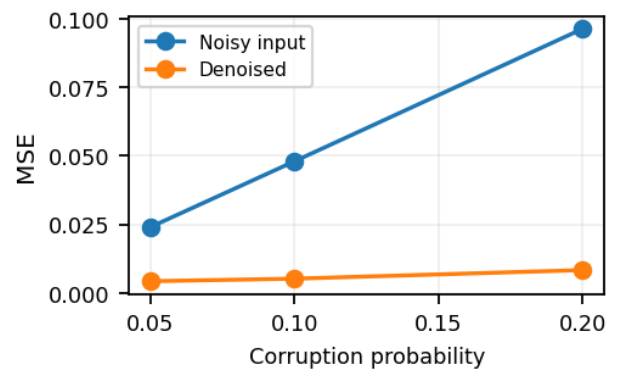
Inference: At Gaussian sigma 0.2, MSE fell from 0.02117 to 0.00449. At salt-and-pepper $p = 0.10$, it fell from 0.04804 to 0.00515. Both models recovered the main digit structure.

7. Noise level and reconstruction quality

Gaussian noise



Salt-and-pepper noise

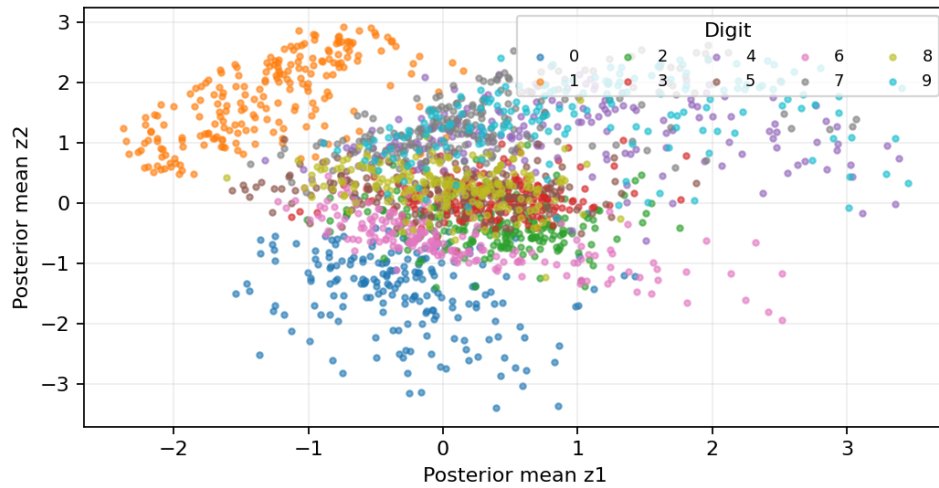


Inference: As Gaussian sigma increased from 0.1 to 0.3, denoised SSIM changed from 0.96153 to 0.86336. More corruption removed useful stroke information.

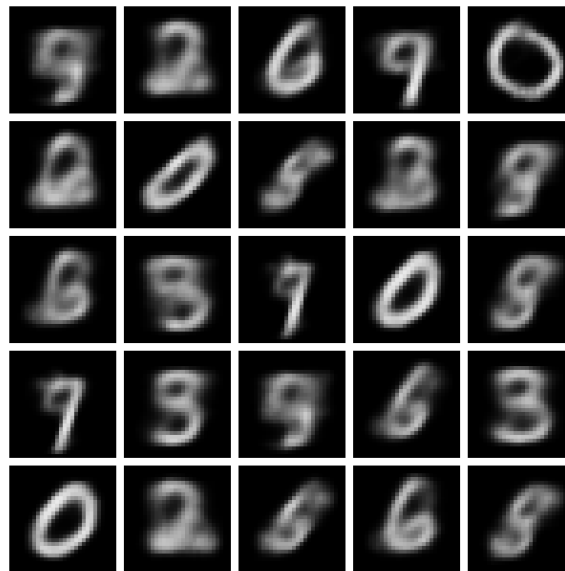
Inference: For salt-and-pepper noise, denoised SSIM changed from 0.95722 at $p = 0.05$ to 0.89960 at $p = 0.20$. The denoised outputs remained better than the noisy inputs.

The same trained model was used across test levels within each noise type. Gaussian standard deviation and salt-and-pepper probability are different measures of corruption.

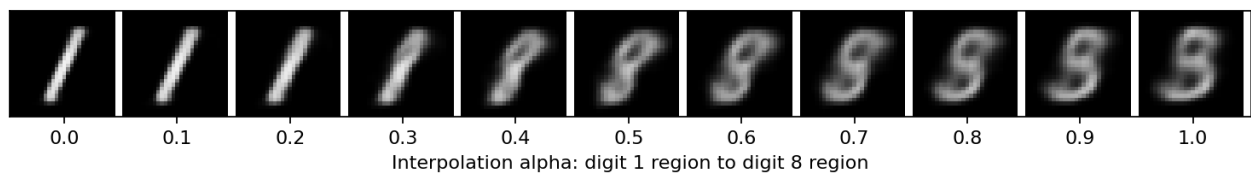
8. VAE latent space and generation



Inference: Some examples of the same digit occupied nearby regions, but several digit groups overlapped. The plotted coordinates are posterior means, not class predictions.



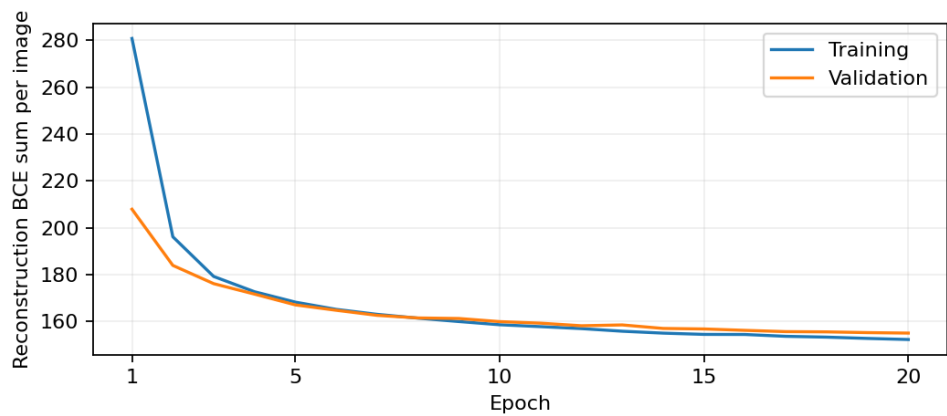
Inference: The 25 random latent samples produced varied digit-like shapes. Some outputs were recognizable, while others were blurred or ambiguous.



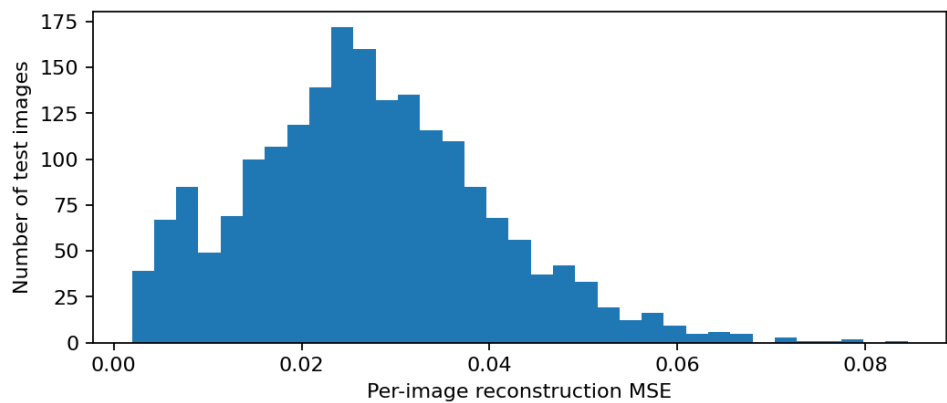
Inference: The decoded shape changed gradually between the selected digit-1 and digit-8 codes. Mean adjacent-image MSE was 0.002041; some intermediate shapes were unclear.

The prior was $N(0, I)$. The VAE learned a distribution for each image, unlike the single code used by the deterministic autoencoders.

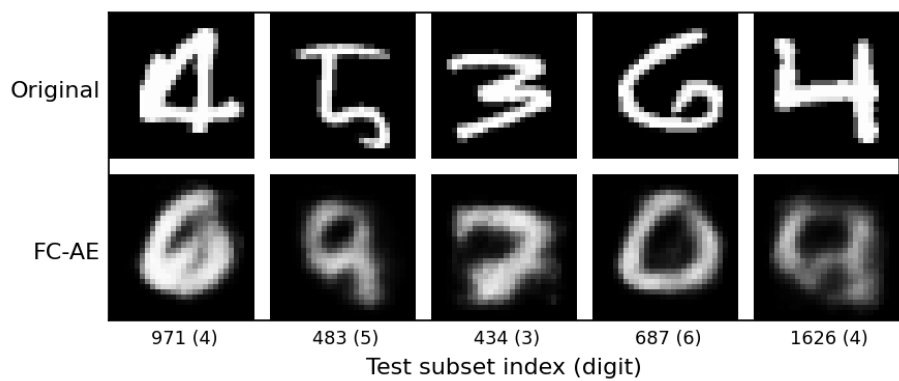
9. VAE loss and high-error images



Inference: VAE validation reconstruction BCE fell from 207.8795 to 155.0317. Reconstruction was tracked separately from KL loss.



Inference: Median FC-AE MSE was 0.02630. The middle half of the images lay between 0.01817 and 0.03543; a smaller set formed the high-error tail.



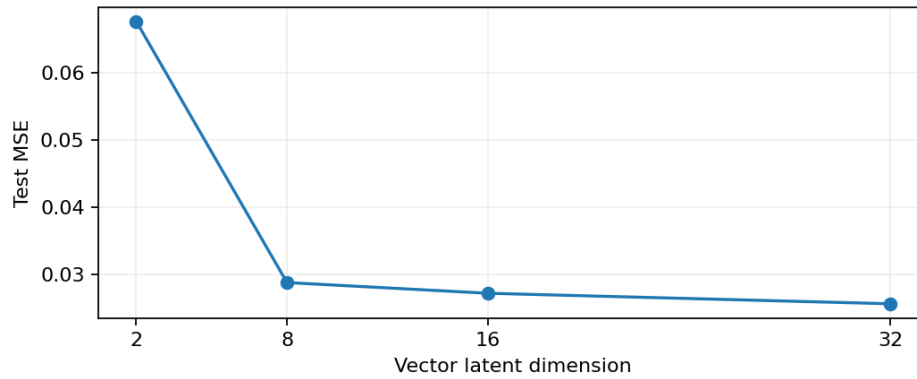
Test subset index	Official test index	Digit	MSE	SSIM
971	5888	4	0.084535	0.427788
483	3778	5	0.078507	0.210007
434	1601	3	0.077531	0.314963
687	8196	6	0.075340	0.399861
1626	3841	4	0.073193	0.325368

Inference: These five images had the largest FC-AE errors. Their reconstructions lost stroke detail or changed the digit shape.

10. Latent-dimension studies

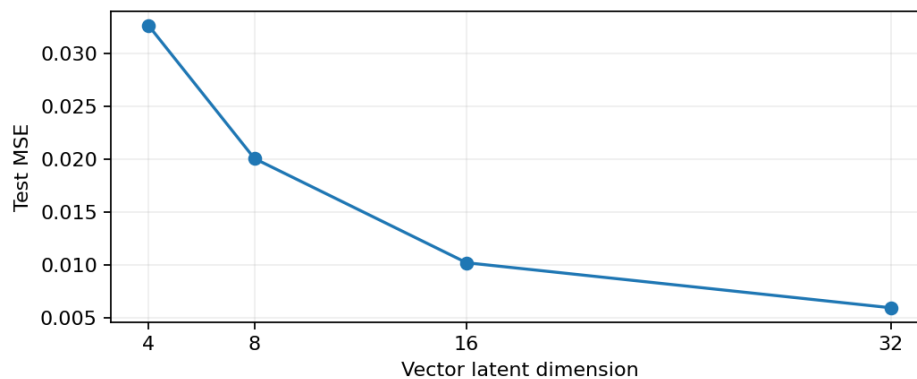
Fully connected vector bottleneck

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC_2	0.067606	0.150665	0.177235	210,130	5.30
FC_8	0.028763	0.072645	0.681657	210,520	5.66
FC_16	0.027172	0.069386	0.692002	211,040	4.51
FC_32	0.025608	0.066786	0.706838	212,080	4.87



Convolutional vector bottleneck

Model	MSE	MAE	SSIM	Parameters	Time (s)
CAE_4	0.032648	0.080543	0.658076	102,725	85.70
CAE_8	0.020067	0.053428	0.789090	127,817	87.09
CAE_16	0.010224	0.033164	0.894051	178,001	87.74
CAE_32	0.005971	0.023702	0.938890	278,369	91.40



Inference: The two-value FC-AE produced zero codes for all 2,000 test images and decoded the same blurred image. Its high error therefore also reflects a training collapse, not just compression.

Inference: The larger FC and convolutional vector bottlenecks gave lower errors in these runs. They retained more detail, but used more parameters and less compression.

11. Additional comparisons

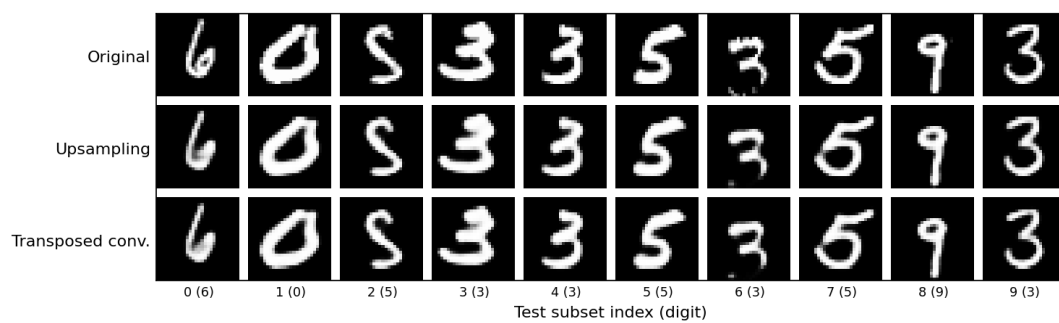
Denoising training level

Model	Test sigma	MSE	MAE	SSIM
DAE_G02	0.10	0.003518	0.017971	0.961530
DAE_G02	0.20	0.004493	0.021158	0.948878
DAE_G02	0.30	0.007470	0.031121	0.863364
DAE_G03	0.10	0.004250	0.019739	0.952989
DAE_G03	0.20	0.004970	0.021821	0.943955
DAE_G03	0.30	0.006552	0.026186	0.923018

Inference: At test sigma 0.3, models trained at sigma 0.2 and 0.3 had SSIM 0.86336 and 0.92302. Training with stronger noise improved recovery at this level.

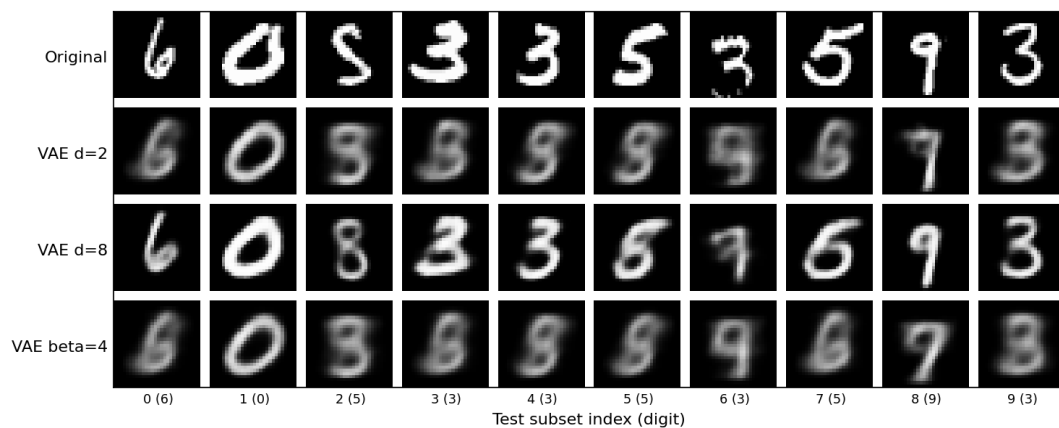
Upsampling versus transposed convolutions

Model	MSE	MAE	SSIM	Parameters	Time (s)
CAE	0.002742	0.015532	0.974080	74,497	76.86
CAE_transpose	0.002303	0.014238	0.977181	89,057	58.95



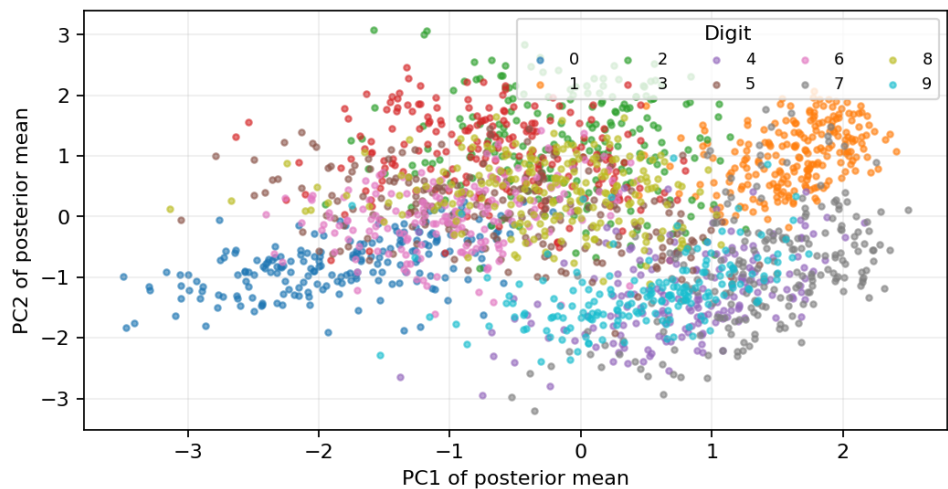
Inference: The transposed-convolution decoder gave MSE 0.00230, compared with 0.00274 for upsampling. It used 14,560 more parameters.

VAE dimension and KL weight

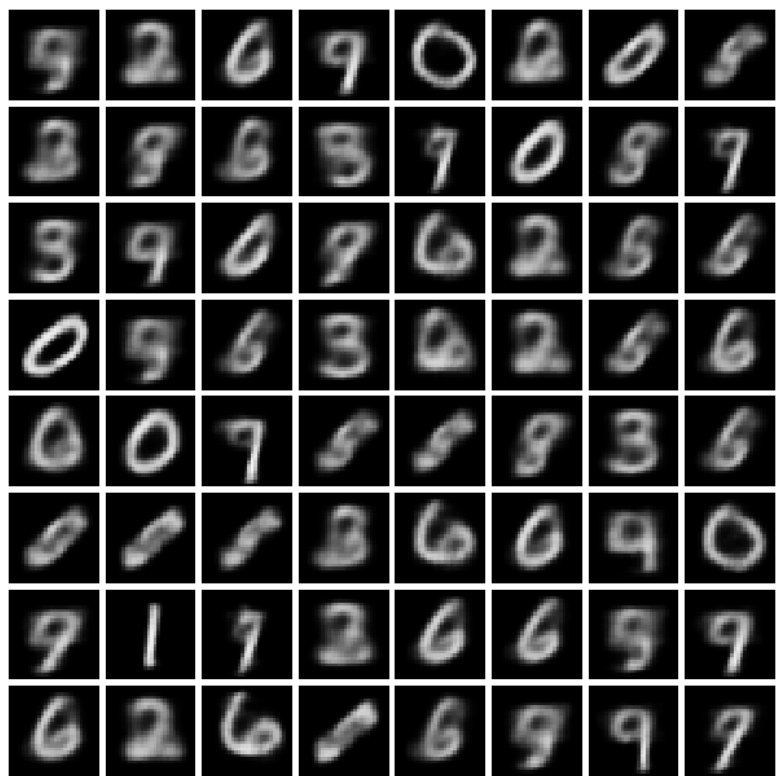


Inference: VAE dimensions 2 and 8 gave MSE 0.04563 and 0.01947. Raising beta from 1 to 4 changed KL from 5.41946 to 3.43387, with less reconstruction detail.

12. Eight-dimensional latent space and more samples



Inference: The two-component PCA view retained 36.71% of the variance in the eight-dimensional posterior means. It shows only a projection of the full code.



Inference: The 64-sample grid showed variation in stroke shape, along with repeated patterns and ambiguous digits. More samples made these strengths and limitations easier to see.

Initial VAE encoder check

Model	MSE	SSIM	KL loss
VAE_2	0.067586	0.174115	0.006636
VAE_8	0.034722	0.627519	10.991966
VAE_beta4	0.057539	0.310198	1.590432

The initial VAE used an extra Dense16/ReLU encoder layer. Its two-dimensional model had no active encoder features on 1,000 checked training images. The final comparisons above removed that extra bottleneck and retrained all three VAE variants.

13. Discussion questions

No.	Question	Answer
1	What is an autoencoder?	A network trained to reconstruct its input.
2	What is the purpose of the encoder?	It converts the image into a learned representation.
3	What is the purpose of the decoder?	It reconstructs the image from the representation.
4	What is a latent representation?	The intermediate code learned by the encoder.
5	Why is it usually smaller than the input?	A smaller code encourages compression instead of simple copying.
6	What happens when it is too small?	Useful detail can be lost and reconstruction error can rise.
7	What happens when its size increases?	It can retain more detail, but compression decreases and improvement is not guaranteed.
8	Why can a CAE preserve spatial information?	Its filters process neighboring pixels and share weights across positions.
9	How does a denoising AE differ?	It receives corrupted inputs but predicts clean targets.
10	Why use a clean denoising target?	The target teaches the network to remove, rather than reproduce, corruption.
11	What happens as noise increases?	Less useful information remains, making recovery harder.
12	Why use MSE, MAE and SSIM?	MSE emphasizes large pixel errors, MAE measures absolute error, and SSIM measures structure.
13	What is a VAE?	A probabilistic encoder-decoder trained with reconstruction and prior-regularization losses.
14	How does it differ from a conventional AE?	It learns a latent distribution instead of only one deterministic code.
15	Why learn a distribution?	A sampleable, regularized representation supports generation and represents variation.
16	What is reparameterization?	It writes a latent sample as mean plus standard deviation times independent noise.
17	Why sample epsilon from $N(0, I)$?	Scaling and shifting standard-normal noise produces the required Gaussian posterior sample.
18	What does KL divergence do?	It penalizes differences between the encoded posterior and the chosen prior.
19	What prior was used?	An independent standard normal distribution, $N(0, I)$.
20	Why use two latent dimensions?	Both coordinates can be plotted directly without dimensionality reduction.
21	What does interpolation demonstrate?	How decoded images change between two latent positions.
22	Why can a VAE generate new images?	Its decoder can map newly sampled prior vectors to image outputs.
23	Why can generated images be unclear?	Compression, regularization, limited training and overlapping patterns can blur shapes.
24	How do reconstruction and generation differ?	Reconstruction encodes a particular image; generation starts from a sampled latent vector.
25	Why keep test images unseen?	They provide an independent measure of generalization.

14. Result

The baseline CAE reconstructed digits more accurately than the 16-value FC-AE. Denoising recovered digit structure from both noise types. The VAE produced new digit-like images and gradual changes between latent codes.

References

[1] TensorFlow, MNIST dataset documentation. MNIST load_data. [2] scikit-image, structural_similarity. [3] Keras, Variational AutoEncoder. [4] CS3807 Experiment 7 laboratory handout.

Appendix A. Recorded epoch values

Epoch	FC train	FC validation	VAE recon. train	VAE recon. validation	VAE KL train	VAE KL validation
1	0.38368	0.26702	280.73367	207.87952	2.37799	2.62354
2	0.26043	0.25035	196.12358	183.91044	3.48542	4.08184
3	0.24272	0.23575	179.18937	176.11941	4.18086	4.24543
4	0.23061	0.22387	172.67368	171.65800	4.33635	4.34040
5	0.21588	0.21027	168.24215	167.06160	4.50927	4.56960
6	0.20361	0.19665	165.14702	164.76116	4.64163	4.81684
7	0.18911	0.18326	163.03607	162.59865	4.68895	4.72708
8	0.17869	0.17639	161.40282	161.49208	4.83315	4.77985
9	0.17280	0.17145	159.95863	161.26005	4.90875	4.89148
10	0.16752	0.16625	158.58597	159.92610	4.99500	5.02581
11	0.16264	0.16212	157.78891	159.27127	5.09849	5.08868
12	0.15910	0.15938	156.94717	158.16218	5.10161	5.03818
13	0.15591	0.15599	155.83078	158.51348	5.22612	5.25622
14	0.15288	0.15279	155.01152	157.01473	5.28763	5.04912
15	0.14991	0.15043	154.48311	156.79576	5.31020	5.34505
16	0.14779	0.14853	154.44177	156.20424	5.28960	5.16089
17	0.14604	0.14731	153.65555	155.65570	5.35477	5.40783
18	0.14472	0.14559	153.30975	155.55459	5.40424	5.33612
19	0.14343	0.14547	152.75775	155.23497	5.43579	5.53291
20	0.14242	0.14401	152.29203	155.03175	5.50327	5.42190

FC losses are BCE per pixel. VAE reconstruction is BCE summed over pixels per image; KL is per image.

Appendix B. Full notebook code

Cell 1

```
import os as oss
import sys as sys
import time as tme
import json as jsn
import hashlib as hsh
import platform as plf
import urllib.request as req
import importlib.metadata as met
from pathlib import Path as Pth
import numpy as npy
import pandas as pds
import torch as tch
from torch import nn as nnn
from torch.nn import functional as fun
import matplotlib.pyplot as plt
from skimage.metrics import structural_similarity as ssm
from IPython.display import display as dsp
tch.set_num_threads(4)
tch.manual_seed(42)
npy.random.seed(42)
tch.use_deterministic_algorithms(True)
dev = tch.device("cpu")
out = Pth("experiment_7_results")
out.mkdir(exist_ok=True)
fig = out / "figures"
fig.mkdir(exist_ok=True)
cfg = {"seed": 42, "train": 10000, "validation": 2000, "test": 2000,
      "epochs": 20, "batch_size": 128, "learning_rate": 0.001,
      "optimizer": "Adam", "device": str(dev), "threads": 4,
      "name": "Joshua", "roll_number": "24110085", "batch": 2,
      "repository": "https://github.com/Joshua-Divide/LAB"}
ver = {nam: met.version(nam) for nam in
      ["torch", "numpy", "pandas", "matplotlib", "scikit-image", "reportlab"]}
cfg["vae_encoder"] = "Conv32 - Conv64 - Flatten - direct mean/log-variance heads"
print(jsn.dumps({"configuration": cfg, "versions": ver}, indent=2))
```

Cell 2

```
def fca(dim):
    enc = nn.Sequential(nnn.Flatten(), nnn.Linear(784, 128), nnn.ReLU(),
                        nnn.Linear(128, 32), nnn.ReLU(),
                        nnn.Linear(32, dim), nnn.ReLU())
    dec = nn.Sequential(nnn.Linear(dim, 32), nnn.ReLU(),
                        nnn.Linear(32, 128), nnn.ReLU(),
                        nnn.Linear(128, 784), nnn.Unflatten(1, (1, 28, 28)))
    return nn.ModuleDict({"enc": enc, "dec": dec})

def cae(dim=None, trn=False):
    enc = [nn.Conv2d(1, 32, 3, padding=1), nnn.ReLU(), nnn.MaxPool2d(2),
           nnn.Conv2d(32, 64, 3, padding=1), nnn.ReLU(), nnn.MaxPool2d(2)]
    dec = []
    if dim is not None:
        enc += [nn.Flatten(), nnn.Linear(3136, dim)]
        dec += [nn.Linear(dim, 3136), nnn.ReLU(),
                nnn.Unflatten(1, (64, 7, 7))]
    dec += [nn.Conv2d(64, 64, 3, padding=1), nnn.ReLU()]
    if trn:
        dec += [nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1),
                nnn.ReLU(), nnn.ConvTranspose2d(32, 1, 4, stride=2, padding=1)]
    else:
        dec += [nn.Upsample(scale_factor=2, mode="nearest"),
                nnn.Conv2d(64, 32, 3, padding=1), nnn.ReLU(),
                nnn.Upsample(scale_factor=2, mode="nearest"),
                nnn.Conv2d(32, 1, 3, padding=1)]
    return nn.ModuleDict({"enc": nnn.Sequential(*enc),
                          "dec": nnn.Sequential(*dec)})

def vae(dim):
    enc = nn.Sequential(nnn.Conv2d(1, 32, 3, stride=2, padding=1), nnn.ReLU(),
                        nnn.Conv2d(32, 64, 3, stride=2, padding=1), nnn.ReLU(),
                        nnn.Flatten())
    dec = nn.Sequential(nnn.Linear(dim, 3136), nnn.ReLU(),
                        nnn.Unflatten(1, (64, 7, 7)),
                        nnn.ConvTranspose2d(64, 64, 4, stride=2, padding=1),
                        nnn.ReLU(),
                        nnn.ConvTranspose2d(64, 32, 4, stride=2, padding=1),
                        nnn.ReLU(), nnn.Conv2d(32, 1, 3, padding=1))
    return nn.ModuleDict({"enc": enc, "mea": nnn.Linear(3136, dim),
                          "log": nnn.Linear(3136, dim), "dec": dec})

def fwd(mod, inp, smp=False, eps=None):
    lat = mod["enc"](inp)
    kld = tch.zeros(len(inp), device=inp.device)
    if "mea" in mod:
        mea, lvr = mod["mea"](lat), mod["log"](lat)
        kld = -0.5 * (1 + lvr - mea.square() - lvr.exp()).sum(1)
        lat = mea
    if smp:
        eps = tch.randn_like(mea) if eps is None else eps
        lat = mea + tch.exp(0.5 * lvr) * eps
    return mod["dec"](lat), kld, lat

def cnt(mod):
    return sum(par.numel() for par in mod.parameters())

arc = []
for nam, mod in [("FC-AE d=16", fca(16)), ("CAE", cae()),
                 ("Denoising CAE", cae()), ("VAE d=2", vae(2))]:
    with tch.no_grad():
        rec, kld, lat = fwd(mod, tch.zeros(2, 1, 28, 28))
        assert tuple(rec.shape) == (2, 1, 28, 28)
    arc.append({"Model": nam, "Parameters": cnt(mod),
               "Latent shape": str(tuple(lat.shape[1:])),
               "Output shape": str(tuple(rec.shape[1:])))})
dsp(pds.DataFrame(arc))
```

Cell 3

```
src = "https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz"
pth = next((pth for pth in [Pth("mnist.npz"), Pth("data/mnist.npz"),
                             Pth.home() / ".keras/datasets/mnist.npz",
                             Pth("/content/mnist.npz")])
            if pth.is_file()), Pth("mnist.npz"))
if not pth.is_file():
    try:
        with req.urlopen(src, timeout=45) as rsp:
            pth.write_bytes(rsp.read())
    except Exception as err:
        raise RuntimeError("MNIST is unavailable. Place the original mnist.npz "
                           "beside this notebook and run all cells again. "
                           "No substitute data or estimated results are used.") from err

dat = npy.load(pth, allow_pickle=False)
assert dat["x_train"].shape == (60000, 28, 28)
assert dat["x_test"].shape == (10000, 28, 28)
assert dat["y_train"].shape == (60000,)
assert dat["y_test"].shape == (10000,)
rng = npy.random.default_rng(cfg["seed"])
idx = rng.permutation(60000)
tix = rng.permutation(10000)[cfg["test"]]
xtr = tch.from_numpy(dat["x_train"][idx[:10000]].astype(npy.float32) / 255)[: , None]
xva = tch.from_numpy(dat["x_train"][idx[10000:12000]].astype(npy.float32) / 255)[: , None]
xts = tch.from_numpy(dat["x_test"][tix].astype(npy.float32) / 255)[: , None]
yts = dat["y_test"][tix]
assert not set(idx[:10000]).intersection(idx[10000:12000])
assert len(npy.unique(tix)) == 2000
cfg["dataset_sha256"] = hsh.sha256(pth.read_bytes()).hexdigest()
np.savez_compressed(out / "split_indices.npz", train=idx[:10000],
                   validation=idx[10000:12000], test=tix)
(out / "configuration.json").write_text(json.dumps({"configuration": cfg,
                                                    "versions": ver}, indent=2))

print("Training:", tuple(xtr.shape), "Validation:", tuple(xva.shape),
      "Test:", tuple(xts.shape))
print("Range:", float(xtr.min()), float(xtr.max()))
print("Test label counts:", npy.bincount(yts, minlength=10))
```

Cell 4

```

def noi(inp, typ, lev, sed):
    gen = tch.Generator().manual_seed(sed)
    if typ == "Gaussian":
        return (inp + lev * tch.randn(inp.shape, generator=gen)).clamp(0, 1)
    if typ == "Salt-pepper":
        rnd = tch.rand(inp.shape, generator=gen)
        return tch.where(rnd < lev / 2, 0,
                        tch.where(rnd > 1 - lev / 2, 1, inp))
    return inp.clone()

def fit(mod, inp, tar, vin, vta, bet=0.0):
    tch.manual_seed(cfg["seed"])
    mod = mod.to(dev).to(memory_format=tch.channels_last)
    opt = tch.optim.Adam(mod.parameters(), lr=cfg["learning_rate"])
    his = []
    beg = tme.perf_counter()
    for epo in range(1, cfg["epochs"] + 1):
        mod.train()
        ord = tch.randperm(len(inp))
        acc = npy.zeros(3, dtype=npy.float64)
        for ids in ord.split(cfg["batch_size"]):
            xxx = inp[ids].to(dev).contiguous(memory_format=tch.channels_last)
            yyy = tar[ids].to(dev)
            opt.zero_grad(set_to_none=True)
            log, kld, lat = fwd(mod, xxx, smp=("mea" in mod))
            rec = fun.binary_cross_entropy_with_logits(log, yyy, reduction="none")
            rec = rec.flatten(1).sum(1) if "mea" in mod else rec.flatten(1).mean(1)
            los = (rec + bet * kld).mean()
            assert tch.isfinite(los)
            los.backward()
            opt.step()
            acc += npy.array([rec.detach().sum().item(),
                            kld.detach().sum().item(),
                            los.detach().item() * len(ids)])

        mod.eval()
        val = npy.zeros(3, dtype=npy.float64)
        gen = tch.Generator().manual_seed(701)
        with tch.no_grad():
            for bix in range(0, len(vin), cfg["batch_size"]):
                ids = slice(bix, bix + cfg["batch_size"])
                xxx = vin[ids].to(dev).contiguous(memory_format=tch.channels_last)
                eps = tch.randn((len(xxx), mod["mea"].out_features), generator=gen) if "mea" in mod else None
                log, kld, lat = fwd(mod, xxx, smp=("mea" in mod), eps=eps)
                rec = fun.binary_cross_entropy_with_logits(log, vta[ids].to(dev), reduction="none")
                rec = rec.flatten(1).sum(1) if "mea" in mod else rec.flatten(1).mean(1)
                val += npy.array([rec.sum().item(), kld.sum().item(),
                                (rec + bet * kld).sum().item()])

        acc /= len(inp)
        val /= len(vin)
        row = {"epoch": epo, "train_rec": acc[0], "train_kl": acc[1],
              "train_total": acc[2], "val_rec": val[0], "val_kl": val[1],
              "val_total": val[2]}
        his.append(row)
        print(f"Epoch {epo:02d}: train={acc[2]:.6f}, validation={val[2]:.6f}", flush=True)
    return mod, pds.DataFrame(his), tme.perf_counter() - beg

def prd(mod, inp):
    mod.eval()
    arr = []
    with tch.no_grad():
        for bat in inp.split(cfg["batch_size"]):
            log, kld, lat = fwd(mod, bat.to(dev).contiguous(memory_format=tch.channels_last))
            arr.append(tch.sigmoid(log).cpu())
    return tch.cat(arr)

def mts(tar, rec):
    xxx, yyy = tar.numpy()[0], rec.numpy()[0]
    err = npy.square(xxx - yyy).mean(axis=(1, 2))
    mae = npy.abs(xxx - yyy).mean(axis=(1, 2))
    sim = npy.array([ssm(aaa, bbb, data_range=1.0, win_size=7)
                     for aaa, bbb in zip(xxx, yyy)])
    return {"MSE": float(err.mean()), "MAE": float(mae.mean()),
           "SSIM": float(sim.mean())}, err, sim

def obj(mod, inp, bet=1.0):
    mod.eval()
    gen = tch.Generator().manual_seed(801)
    tot = npy.zeros(2, dtype=npy.float64)
    with tch.no_grad():
        for bat in inp.split(cfg["batch_size"]):
            xxx = bat.to(dev).contiguous(memory_format=tch.channels_last)
            eps = tch.randn((len(xxx), mod["mea"].out_features), generator=gen)
            log, kld, lat = fwd(mod, xxx, smp=True, eps=eps)
            rec = fun.binary_cross_entropy_with_logits(log, xxx, reduction="none").flatten(1).sum(1)
            tot += npy.array([rec.sum().item(), kld.sum().item()])
    tot /= len(inp)
    return {"Reconstruction loss": float(tot[0]), "KL loss": float(tot[1]),
           "Total loss": float(tot[0] + bet * tot[1])}

mds, hst, tms, prs, ers, sms, rws = {}, {}, {}, {}, {}, {}
def run(nam, mod, typ="Clean", lev=0.0, bet=0.0):
    print("Training", nam, flush=True)
    inp = noi(xtr, typ, lev, 101)
    vin = noi(xva, typ, lev, 201)
    mod, his, sec = fit(mod, inp, xtr, vin, xva, bet)

```

```

rec = prd(mod, noi(xts, typ, lev, 301))
met, err, sim = mts(xts, rec)
row = {"Model": nam, "Input": typ, "Level": lev, **met,
      "Parameters": cnt(mod), "Time (s)": sec}
if "mea" in mod:
    row.update(obj(mod, xts, bet))
mds[nam], hst[nam], tms[nam], prs[nam] = mod, his, sec, rec
ers[nam], sms[nam] = err, sim
rws.append(row)
his.to_csv(out / (nam + "_history.csv"), index=False)
tch.save(mod.state_dict(), out / (nam + ".pt"))
pds.DataFrame(rws).to_csv(out / "results.csv", index=False)
print(json.dumps(row, indent=2, flush=True))

```

Cell 5

```

for dim in [2, 8, 16, 32]:
    tch.manual_seed(42)
    run(f"FC_{dim}", fca(dim))

```

Cell 6

```

tch.manual_seed(42)
run("CAE", cae())
for dim in [4, 8, 16, 32]:
    tch.manual_seed(42)
    run(f"CAE_{dim}", cae(dim))
tch.manual_seed(42)
run("CAE_transpose", cae(trn=True))

```

Cell 7

```

for nam, typ, lev in [("DAE_G02", "Gaussian", 0.2),
                     ("DAE_G03", "Gaussian", 0.3),
                     ("DAE_SP10", "Salt-pepper", 0.1)]:
    tch.manual_seed(42)
    run(nam, cae(), typ, lev)

```

Cell 8

```

for nam, dim, bet in [("VAE_2", 2, 1.0), ("VAE_8", 8, 1.0),
                     ("VAE_beta4", 2, 4.0)]:
    tch.manual_seed(42)
    run(nam, vae(dim), bet=bet)
res = pds.DataFrame(rws).set_index("Model")
dsp(res.round(6))

```

Cell 9

```

nrs = []
for nam, typ, lev in [("DAE_G02", "Gaussian", lev) for lev in [0.1, 0.2, 0.3]] +
["DAE_SP10", "Salt-pepper", lev) for lev in [0.05, 0.1, 0.2]]:
    inp = noi(xts, typ, lev, 301)
    for tag, rec in [("Noisy input", inp), ("Denoised", prd(mds[nam], inp))]:
        met, err, sim = mts(xts, rec)
        nrs.append({"Noise": typ, "Level": lev, "Output": tag, **met})
nrs = pds.DataFrame(nrs)
nrs.to_csv(out / "noise_results.csv", index=False)
dsp(nrs.round(6))
trs = []
for nam in ["DAE_G02", "DAE_G03"]:
    for lev in [0.1, 0.2, 0.3]:
        met, err, sim = mts(xts, prd(mds[nam], noi(xts, "Gaussian", lev, 301)))
        trs.append({"Model": nam, "Test sigma": lev, **met})
trs = pds.DataFrame(trs)
trs.to_csv(out / "denoising_training_levels.csv", index=False)
dsp(trs.round(6))
cln = []
for nam in ["FC_16", "CAE", "DAE_G02", "VAE_2"]:
    met, err, sim = mts(xts, prd(mds[nam], xts))
    cln.append({"Model": nam, **met})
cln = pds.DataFrame(cln)
cln.to_csv(out / "clean_input_comparison.csv", index=False)
dsp(cln.round(6))

```

Cell 10

```

from IPython.display import Image as Ipi
def sav(nam):
    plt.tight_layout()
    plt.savefig(fig / (nam + ".png"), dpi=160, bbox_inches="tight")
    plt.close()
    dsp(Ipi(filename=str(fig / (nam + ".png"))))
def grd(arr, lab, nam, num=10, ids=None):
    ids = npy.arange(num) if ids is None else npy.asarray(ids)
    can = npy.ones((len(arr) * 31 - 3, len(ids) * 31 - 3))
    for rid, mat in enumerate(arr):
        mat = mat.numpy()[:, 0] if isinstance(mat, tch.Tensor) else mat
        for cid, idx in enumerate(ids):
            can[rid * 31:rid * 31 + 28, cid * 31:cid * 31 + 28] = mat[idx]
    plt.figure(figsize=(9.4, 0.9 * len(arr) + 0.7))
    plt.imshow(can, vmin=0, vmax=1, cmap="gray", interpolation="nearest")
    plt.yticks([idx * 31 + 14 for idx in range(len(arr))], lab)
    plt.xticks([idx * 31 + 14 for idx in range(len(ids))],
               [f"{idx} ({yts[idx]})" for idx in ids], fontsize=8)
    plt.xlabel("Test subset index (digit)")
    plt.tick_params(length=0)
    sav(nam)
def crv(nam, met, ttl, out):
    tab = hst[nam]
    plt.figure(figsize=(6.7, 3.0))
    plt.plot(tab["epoch"], tab["train_" + met], label="Training")
    plt.plot(tab["epoch"], tab["val_" + met], label="Validation")
    plt.xlabel("Epoch")
    plt.ylabel(ttl)
    plt.xticks([1, 5, 10, 15, 20])
    plt.legend()
    plt.grid(alpha=0.2)
    sav(out)
def lat(nam):
    mod = mds[nam]
    arr = []
    with tch.no_grad():
        for bat in xts.split(128):
            enc = mod["enc"](bat.to(dev))
            arr.append((mod["mea"](enc) if "mea" in mod else enc).cpu())
    return tch.cat(arr).numpy()
def sca(arr, nam, xlb, ylb):
    plt.figure(figsize=(6.7, 3.6))
    for dig in range(10):
        idx = yts == dig
        plt.scatter(arr[idx, 0], arr[idx, 1], s=8, alpha=0.6, label=str(dig))
    plt.xlabel(xlb)
    plt.ylabel(ylb)
    plt.legend(ncol=5, fontsize=8, title="Digit", loc="upper right")
    plt.grid(alpha=0.2)
    sav(nam)
def gen(mod, num, nam):
    rng = tch.Generator().manual_seed(901)
    zzz = tch.randn(num * num, mod["mea"].out_features, generator=rng)
    with tch.no_grad():
        arr = tch.sigmoid(mod["dec"](zzz.to(dev))).cpu().numpy()[:, 0]
    can = npy.ones((num * 30 - 2, num * 30 - 2))
    for idx, img in enumerate(arr):
        rid, cid = divmod(idx, num)
        can[rid * 30:rid * 30 + 28, cid * 30:cid * 30 + 28] = img
    plt.figure(figsize=(4.0, 4.0))
    plt.imshow(can, vmin=0, vmax=1, cmap="gray", interpolation="nearest")
    plt.axis("off")
    sav(nam)
    return arr

```

Cell 11

```

grd([xts, prs["FC_16"], (xts - prs["FC_16"]).abs()],
    ["Original", "FC-AE", "Absolute error"], "01_basic")
crv("FC_16", "rec", "Binary cross-entropy per pixel", "02_fc_loss")
grd([xts, prs["FC_16"], prs["CAE"]],
    ["Original", "FC-AE", "CAE"], "03_comparison")
grd([xts, noi(xts, "Gaussian", 0.2, 301), prs["DAE_G02"]],
    ["Clean", "Gaussian 0.2", "Denoised"], "04_gaussian")
grd([xts, noi(xts, "Salt-pepper", 0.1, 301), prs["DAE_SP10"]],
    ["Clean", "Salt-pepper 0.1", "Denoised"], "05_salt_pepper")
for typ, tag in [("Gaussian", "gaussian"), ("Salt-pepper", "salt_pepper")]:
    for met in ["MSE", "MAE", "SSIM"]:
        plt.figure(figsize=(3.35, 2.1))
        for lbl in ["Noisy input", "Denoised"]:
            tab = nrs[(nrs["Noise"] == typ) & (nrs["Output"] == lbl)]
            plt.plot(tab["Level"], tab[met], marker="o", label=lbl)
        plt.xlabel("Gaussian standard deviation" if typ == "Gaussian" else "Corruption probability",
            fontsize=8)
        plt.ylabel(met, fontsize=9)
        plt.xticks(fontsize=8)
        plt.yticks(fontsize=8)
        plt.legend(fontsize=7)
        plt.grid(alpha=0.2)
        sav("noise_" + tag + "_" + met)

```

Cell 12

```

l02, l08 = lat("VAE_2"), lat("VAE_8")
sca(l02, "06_latent_2", "Posterior mean z1", "Posterior mean z2")
avg = l08.mean(0)
uuu, svd, vvv = npy.linalg.svd(l08 - avg, full_matrices=False)
pcv = (svd[:2] ** 2).sum() / (svd ** 2).sum()
sca((l08 - avg) @ vvv[:2].T, "latent_8_pca", "PC1 of posterior mean", "PC2 of posterior mean")
r25 = gen(mds["VAE_2"], 5, "07_samples_25")
r64 = gen(mds["VAE_2"], 8, "samples_64")
ids = [int(npy.flatnonzero(yts == dig)[0]) for dig in [1, 8]]
alp = npy.linspace(0, 1, 11)
zzz = (1 - alp[:, None]) * l02[ids[0]] + alp[:, None] * l02[ids[1]]
with tch.no_grad():
    itr = tch.sigmoid(mds["VAE_2"]["dec"](tch.tensor(zzz, dtype=tch.float32))).cpu().numpy()[:, 0]
can = npy.concatenate([npy.pad(img, ((0, 0), (0, 2)), constant_values=1)
                        for img in itr], axis=1)
plt.figure(figsize=(9.4, 1.4))
plt.imshow(can, vmin=0, vmax=1, cmap="gray", interpolation="nearest")
plt.xticks([idx * 30 + 14 for idx in range(11)], [f"{val:.1f}" for val in alp])
plt.yticks([])
plt.xlabel("Interpolation alpha: digit 1 region to digit 8 region")
sav("08_interpolation")
crv("VAE_2", "rec", "Reconstruction BCE sum per image", "09_vae_loss")
crv("VAE_2", "kl", "KL divergence per image", "vae_kl_loss")
print("VAE d=2 posterior mean average:", l02.mean(0))
print("VAE d=2 posterior mean standard deviation:", l02.std(0))
print("VAE d=8 PCA variance retained:", pcv)
print("Interpolation test indices:", ids)
print("Interpolation endpoints:", l02[ids])
print("Mean adjacent interpolation MSE:", npy.square(npy.diff(itr, axis=0)).mean())

```

Cell 13

```

plt.figure(figsize=(6.7, 3.0))
plt.hist(ers["FC_16"], bins=35)
plt.xlabel("Per-image reconstruction MSE")
plt.ylabel("Number of test images")
sav("10_error_distribution")
hiq = npy.argsort(ers["FC_16"])[-5:][:-1]
grd([xts, prs["FC_16"]], ["Original", "FC-AE"], "high_error", ids=hiq)
her = pds.DataFrame({"Test subset index": hiq, "Official test index": tix[hiq],
                    "Digit": yts[hiq], "MSE": ers["FC_16"][hiq],
                    "SSIM": sms["FC_16"][hiq]})
her.to_csv(out / "high_error_samples.csv", index=False)
dsp(her.round(6))
qnt = npy.quantile(ers["FC_16"], [0, 0.05, 0.25, 0.5, 0.75, 0.95, 1])
print("FC MSE min, P5, P25, median, P75, P95, max:", qnt)
for fam, dim in [{"FC", [2, 8, 16, 32]}, {"CAE", [4, 8, 16, 32]}]:
    tab = res.loc[[f"{fam}_{val}" for val in dim]]
    plt.figure(figsize=(6.7, 2.9))
    plt.plot(dim, tab["MSE"], marker="o")
    plt.xlabel("Vector latent dimension")
    plt.ylabel("Test MSE")
    plt.xticks(dim)
    plt.grid(alpha=0.2)
    sav("latent_dimension_" + fam)
    dsp(tab[["MSE", "MAE", "SSIM", "Parameters", "Time (s)"]].round(6))
grd([xts, prs["CAE"], prs["CAE_transpose"]],
    ["Original", "Upsampling", "Transposed conv.", "decoder_comparison"])
grd([xts, prs["VAE_2"], prs["VAE_8"], prs["VAE_beta4"]],
    ["Original", "VAE d=2", "VAE d=8", "VAE beta=4"], "vae_comparison")
zero_rows = []
for dimension in [2, 8, 16, 32]:
    code = lat(f"FC_{dimension}")
    zero_rows.append({"Latent dimension": dimension,
                     "Zero-code test images": int(npy.all(code == 0, axis=1).sum()),
                     "Inactive coordinates": int(npy.all(code == 0, axis=0).sum())})
zero_codes = pds.DataFrame(zero_rows)
zero_codes.to_csv(out / "latent_activity.csv", index=False)
dsp(zero_codes)

```

Cell 14

```

man = ["FC_16", "CAE", "DAE_G02", "VAE_2"]
con = res.loc[man, ["Input", "Level", "MSE", "MAE", "SSIM", "Parameters", "Time (s)"]]
dsp(con.round(6))
dsp(res.loc[["VAE_2", "VAE_8", "VAE_beta4"],
            ["Reconstruction loss", "KL loss", "Total loss", "MSE", "MAE", "SSIM"]].round(6))
epo = hst["FC_16"][["epoch", "train_rec", "val_rec"]].copy()
epo.columns = ["Epoch", "FC train BCE", "FC validation BCE"]
epo["VAE train reconstruction"] = hst["VAE_2"]["train_rec"]
epo["VAE validation reconstruction"] = hst["VAE_2"]["val_rec"]
epo["VAE train KL"] = hst["VAE_2"]["train_kl"]
epo["VAE validation KL"] = hst["VAE_2"]["val_kl"]
epo.to_csv(out / "epoch_values.csv", index=False)
dsp(epo.round(6))
assert len(res) == 16
assert all(len(tab) == 20 for tab in hst.values())
assert npy.isfinite(res[["MSE", "MAE", "SSIM", "Parameters", "Time (s)"]].to_numpy()).all()
assert all(tuple(arr.shape) == (2000, 1, 28, 28) for arr in prs.values())
assert len(nrs) == 12 and len(trs) == 6
print("Completed:", len(res), "trained models; all expected result tables are populated.")
summary = {"architecture": arc, "fc_error_quantiles": qnt.tolist(),
           "vae_mean_average": l02.mean(0).tolist(),
           "vae_mean_standard_deviation": l02.std(0).tolist(),
           "vae_8_pca_variance": float(pcv),
           "interpolation_indices": ids,
           "interpolation_endpoints": l02[ids].tolist(),
           "adjacent_interpolation_mse": float(npy.square(npy.diff(itr, axis=0)).mean()),
           "model_count": len(res), "epochs_per_model": cfg["epochs"]}
(out / "analysis_summary.json").write_text(json.dumps(summary, indent=2))
per_image = pds.DataFrame({"Subset index": npy.arange(len(xts)),
                           "Official test index": tix, "Digit": yts})
for model_name in res.index:
    per_image[model_name + " MSE"] = ers[model_name]
    per_image[model_name + " SSIM"] = sms[model_name]
per_image.to_csv(out / "per_image_metrics.csv", index=False)
npz.savez_compressed(out / "vae_analysis.npz", latent_2=l02, latent_8=l08,
                    generated_25=r25, generated_64=r64, interpolation=itr,
                    alpha=alp, labels=yts)

```